Supplementary Material for "ACaaS: Privacy-Preserving Access Control as a Service for Healthcare on the Cloud–Edge Continuum"

Ha Xuan Son[1⋆], Phat T. Tran-Truong[2,3⋆], Bach Le-Xuan[2,3⋆⋆], Minh B. Nguyen[4], and Thanh Pham[5]

[1] Digital Economy, The Business School, SGS Campus, RMIT University Vietnam, Ho Chi Minh City, Vietnam
ha.son@rmit.edu.vn
[2] Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), Ho Chi Minh City, Vietnam
[3] Vietnam National University Ho Chi Minh City (VNU-HCM), Ho Chi Minh City, Vietnam
phatttt@hcmut.edu.vn; lexuanbach@hcmut.edu.vn
[4] Can Tho University of Technology, Can Tho City, Vietnam
nbminh@ctuet.edu.vn
[5] School of Science, Engineering and Technology, SGS Campus, RMIT University Vietnam, Ho Chi Minh City, Vietnam
thanh.pham@rmit.edu.vn
[6]

Abstract. This supplement provides material referenced from the main paper: a point-by-point summary of the changes with respect to the ICSOC 2026 early submission (Sect. 1); detailed experimental setup covering hardware specifications, synthetic workload generation, network emulation parameters, baseline configurations, and per-experiment parameters for all twelve experiments E1–E12 (Sect. 2); expanded proofs of Lemma 1, Theorems 1–5, Corollary 1, and Proposition 1 (Sect. 3); the step-numbered policy-synchronization diagram (Sect. 4); the Data Minimizer pipeline diagram (Sect. 5); distribution plots for experiments E1/E4/E5 and companion plots for E8/E11/E12 (Sect. 6); the per-site E12 soak table and the real-Fabric transcript summary (Sect. 7); and reproduction instructions (Sect. 8). Reviewers are not required to consult this material; all claims of the main paper are supported there.

1 Changes with Respect to the ICSOC 2026 Early Submission (Paper 81)

This section summarizes, point by point, how the resubmission addresses the early-submission reviews; it mirrors the accompanying response letter. Section, theorem, equation, and figure numbers refer to the main paper.

⋆ These authors contributed equally to this work.
⋆⋆ Corresponding author.

Meta-review / Reviewer 1 — architecture too abstract; security goals not validated; evaluation only performance. Section 4.1 now specifies the typed inter-service contracts C1–C3, a numbered happy-path sequence, and failure paths F1–F3; Sect. 4.2 formalizes the three-channel design, the chaincode invariant $\mathsf{INV}_{\text{mono}}$ (Eq. 2), and the multi-org endorsement rule. The new Sect. 4.5 discharges G1–G5 via Lemma 1, Theorems 1–5, Proposition 1, and Corollary 1. Crucially, the evaluation no longer rests on performance alone: Sect. 5.2 adds experiments E6–E12 that mechanize every theorem as executable assertions or measurements—a 21-assertion conformance suite, an analytic-model validation (R^2=0.998), a Minimizer cost profile, an empirical re-identification attack within one percentage point of the $1/k$ bound, a ledger-vs-pub/sub tamper ablation, Byzantine fault injection (f=1) with zero safety violations, and a three-site partition/soak emulation.

Reviewer 2 — healthcare specificity and regulations unclear. The Introduction anchors each design constraint to named clauses: HIPAA §164.312(a)(1)–(d) and (b), GDPR Articles 9, 25 (edge-origin minimization "by design"), 30 (LogChain/PolicyChain records of processing), and 32; break-glass access, rural-clinic and ambulance partitions, and revocation/outbreak-driven policy pushes are named as the operational drivers. Theorems 3 and 5 tie the guarantees back to these clauses.

Reviewer 2 — related work not strong enough. Section 2 was rebuilt around the three directly comparable categories, closes with an explicit positioning paragraph, and adds Table 1 comparing ACaaS to ten systems on six axes; the closest competitor (QuickMedBlock) is analyzed by name.

Reviewer 2 — validation should move closer to real deployment. Three additions: (i) the new E12 twelve-hour, three-site partition/soak emulation (27,360 requests, 37 partitions, 100% local availability, zero log loss); (ii) a sanity check on a *real* Hyperledger Fabric test network confirming the latency constants assumed by the analytic model; (iii) a one-command reproducible artefact. A partner-hospital shadow deployment is scoped as future work (Limitations, Sect. 5.2).

Reviewer 2 — Figures 1 and 3 text too small. The architecture figure was redrawn natively in Ti*k*Z at LNCS column width (Fig. 1); no in-figure label falls below approximately 7 pt. The former policy-synchronization figure is replaced by an explicit Phase A/B/C specification in Sect. 4.2 plus the redrawn step-numbered diagram in Sect. 4 of this supplement.

Reviewer 2 — ICSOC fit unclear. The opening of the Introduction frames multi-site healthcare as a service-composition problem; contribution (i) names the targeted themes (service composition, resilient service delivery on the cloud–edge continuum); the contracts C1–C3 are the explicit unit of composition. The work aligns with Focus Areas 3 (cloud–edge continuum, XaaS) and 4 (blockchain and cybersecurity services).

Reviewer 3 — related work too broad. Tangential surveys (TinyML biometrics, general CDSS, AI-supported explainability) were removed; the section now covers only the access-control decision pipeline.

Reviewer 3 — side-by-side experimental presentation. All experiments are consolidated into Table 2 plus two multi-panel figures with uniform axes and labels: Fig. 2 (E2/E3 curves) and Fig. 3 (E7/E9/E10); companion plots for E1/E4/E5 and E8/E11/E12 appear in Sect. 6 with identical styling.

Reviewer 3 — novelty is combination of known techniques; claims lack (semi-)formal backing. We no longer claim novelty in any individual primitive. The contribution is the *composition* and its provable properties: substitution-resistant synchronization (Lemma 1) that demonstrably disappears under component replacement (E10), eventual consistency with a closed-form reconciliation bound (Theorem 1, Corollary 1), knapsack-near-optimal tier placement (Proposition 1) validated at R^2=0.998 (E7), and an enforced no-raw-egress invariant (Theorem 3) calibrated by an actual attack (E9). Every formal statement now has an executable counterpart; machine-checked proofs remain future work and are stated as such.

2 Detailed Experimental Setup and Workloads

This section provides the comprehensive evaluation methodology requested by reviewers, covering hardware specifications, workload generation, network emulation parameters, baseline configurations, and per-experiment settings for all twelve experiments (E1–E12).

Testbed Hardware

Cloud tier. A high-performance local server simulating a cloud instance, running Ubuntu 22.04 LTS with an Intel Core i7-12700 CPU (12 cores, 2.1 GHz base), 32 GB DDR5 RAM, and a 1 TB NVMe SSD. This host runs: the Policy Manager Service (PAP), the full OPA-based cloud PDP, the API Gateway (Nginx with OIDC middleware), the Audit & Compliance service, and a four-peer Hyperledger Fabric network (two organizations, Raft ordering, f=1 fault tolerance). All Fabric peers are co-located on the same physical host via Docker, so inter-peer network overhead is minimal (loopback); the Real-Fabric transcript in Sect. 7 provides a sanity check of the latency constants on a geographically distributed Fabric test network.
Fog tier. A mid-range desktop running Ubuntu 22.04 LTS with an Intel Core i5-10400 CPU (6 cores, 2.9 GHz), 16 GB DDR4 RAM. This host runs the Policy Cache (TTL-based invalidation, version-keyed) and the Log Aggregation service.
Edge tier. A Raspberry Pi 4 Model B (ARM Cortex-A72, 4 GB RAM) running Raspberry Pi OS 64-bit (Debian 11 base), representing a rural clinic gateway or ambulance mobile unit. This host runs the lightweight OPA-WASM PDP, the Local Policy Store, the Data Minimizer, and the Offline Queue (disk-backed, monotone-sequenced).

All three hosts reside on the same physical LAN; WAN latency and partitions are *emulated* via Linux `tc-netem` on the network interface connecting the cloud and fog/edge hosts, as described below.

Network Emulation

WAN conditions (E1, E5, E12). A fixed 50 ms round-trip latency is imposed between cloud and edge using `tc-netem add delay 25ms` on both ends, approximating a rural hospital-to-datacenter WAN link. Bandwidth is capped at 10 Mbit/s, matching common rural broadband constraints.
Partition injection (E5, E12). WAN partitions are emulated by applying `tc-netem loss 100%` on the cloud-facing interface of the edge host for a programmatically controlled duration. The edge enters and exits OFFLINE mode autonomously based on heartbeat failure/recovery detection (3-second timeout). For E12, partitions are injected at random intervals drawn from $U[2, 8]$ minutes per site profile.

Synthetic EHR Workload Generation

No real patient data was used. Synthetic EHR records were generated procedurally using a schema derived from the HL7 FHIR R4 Patient and Observation resources:

- **Date of birth**: uniformly sampled, ages 19–74.
- **Postal code**: 200 synthetic codes with Zipf-distributed frequency (emulating urban hot spots).
- **Gender**: binary, 50/50. **Ethnicity**: four categories, uniform.
- **Sensitive attribute**: ICD-10 diagnosis code, lab result flag, or medication class (uniform).
- **Record sizes**: small ≈512 B (vitals only), medium ≈2 KB (full admission), large ≈8 KB (imaging metadata).

Policy Corpus

200 synthetic ABAC rules cover the most common clinical access patterns: role-based EHR read, break-glass emergency access, research de-identification, audit-log write, and formulary check. Policies are compiled from YAML source to OPA Rego bundles; compiled sizes range from 1.8 KB to 6.9 KB (mean 3.4 KB, factor $<4\times$ spread, consistent with Proposition 1). Each policy is signed with ECDSA-P256; the signature is included in the committed ledger tuple.

Per-Experiment Parameters

E1 (Latency, G4). 1 000 sequential requests from a single client; 50 ms WAN active; policies pre-loaded. Metric: median and IQR of end-to-end decision latency.
E2 (Throughput, G4/G2). Concurrent load swept across six levels (50, 100, 200, 500, and 1 000 workers; Python `ThreadPoolExecutor`), each worker issuing 20 requests (up to 20 000 total). Metric: sustained throughput (req/s), P95 response time, zero-failure threshold enforced.

E3 (Sync consistency, G1/Corollary 1). Policy-batch commit to PolicyChain; synchronization latency measured to last-edge ACK. Batch sizes: 1, 5, 10, 25, 50, 100, and 200 policies (seven levels); 10 repetitions each. Metric: median ± IQR per batch size.

E4 (Ledger overhead, G2/G5). *Baseline*: direct PostgreSQL write (tamper-evident audit log, no blockchain). *ACaaS*: Hyperledger Fabric endorsement via the same gRPC client. 200 operations per configuration. Metric: per-operation latency boxplot. The Fabric overhead quantifies the cost of the tamper-evidence guarantee that the direct-DB baseline cannot provide.

E5 (Offline resilience, G4). WAN severed for 60 s; edge serves 1 000 requests from cache. Connectivity restored; Offline Queue drained. Metric: online vs. offline latency distribution; zero drain failures.

E6 (Conformance suite). Full 33-test pytest suite (20 unit tests + 5 blockchain mock tests + 8 cross-validation tests), no service mocking (Docker Compose stack). The 21 security assertions cover: eventual consistency (50-update offline burst, 10 trials); $\mathsf{INV}_{\text{mono}}$ enforcement (5 rollback attempts rejected); F3 hash-mismatch rejection (20 injected tampered bodies); seq_e gap detection (5 dropped records); drain-without-loss (50 queued records); no-direct-identifier-egress (verified via `grep`-based import-level check that no module reachable from the edge PDP response path imports `requests`, `httpx`, or `urllib` + runtime trace); k-anonymity cohort check (50 batches); import-level no-WAN-coupling of edge PDP (confirmed: `edge/lightweight-pdp/` has no WAN-touching dependency); audit-replay reconstruction end-to-end. Pass threshold: 100%.

E7 (Cache hit-rate, Proposition 1). ρ_{edge} swept across nine evenly-spaced levels from 0.10 to 1.00 (step 0.15) by controlling the pre-loaded policy fraction; 1 000 samples per setting. Predicted latency uses measured constants $T_{\text{edge}} = 0.393\,\text{ms}$, $T_{\text{WAN}} = 50\,\text{ms}$, $T_{\text{cloud}} = 7.81\,\text{ms}$.

E8 (Minimizer cost, G3). 1 000 records per size class (small/medium/large) through all three pipeline stages; 10 warm-up runs discarded; timing via `perf_counter_ns`. k-anonymity scaling: 5 000 records per cohort at $k \in \{5, 10, 20\}$.

E9 (Re-identification attack, Theorem 3). Adversary: honest-but-curious clinician with exact quasi-identifier background knowledge. Cohort: 10 000 synthetic records; 2 000 attack trials per (k, R); $k \in \{5, 10, 20\}$ and $R = 1, \ldots, 10$ independent releases from disjoint cohorts. Baseline: $1/k$ single-release bound and cumulative $1 - (1 - 1/k)^R$.

E10 (Substrate ablation, Lemma 1). Config A: ACaaS ledger (PolicyChain hash-anchor). Config B: Redis 7 Pub/Sub baseline (no body-to-hash anchoring; Redis 7-alpine running as a Docker container on the cloud host, co-located with the Fabric peers). Subscriber count swept across five levels: 1, 5, 10, 25, and 50. Tamper: policy body replaced en route while ledger entry unchanged. Metric: propagation latency and tamper-detection rate (expected: 100% ledger, 0% pub/sub).

E11 (Byzantine fault, f=1). Four-peer ensemble, `AND(Org1,Org2)` threshold. One peer runs: (a) passive, (b) rewind (signs outdated version), (c) fork (signs

different body). 50 commit attempts per scenario (200 total). Metrics: liveness rate and $\mathsf{INV}_{\text{mono}}$ violation count.

E12 (Three-site soak, G1/G2/G4). 720 simulated minutes (6× time compression, achieved by scaling all sleep and timer intervals by 1/6 in the emulation harness, so the test completes in ≈ 120 real minutes); three site profiles:

- *Urban hospital ward gateway*: 18 req/min, 4 partitions (mean 5 min each).
- *District clinic*: 12 req/min, 12 partitions (mean 8 min each).
- *Mobile unit*: 8 req/min, 21 partitions (mean 12 min each).

Workload rates represent the authorization traffic routed through a *single edge gateway* serving one department or vehicle; aggregate hospital-wide traffic is covered by the throughput sweep in E2. 29 policy updates at random intervals (seed 20260704). Metrics: per-site availability, offline/stale decision counts, LogChain entry count vs. zero-loss threshold, peak Offline Queue backlog, maximum missed updates before Phase C reconciliation. Results in Table 1.

3 Expanded Proofs

Throughout, Σ is the signature scheme (EUF-CMA secure), H = SHA-256 (second-preimage resistant), and the Fabric ordering service is safe with fewer than f Byzantine orderers (f=1 of four in our configuration). Numbering follows the main paper.

Lemma 1 (Substitution resistance)

Statement. Let (id, v, h, σ) be committed on PolicyChain. The probability that an honest edge executing Phase B or C accepts a body $\pi' \neq \pi$ for this entry is at most $\mathbf{Adv}_H^{\text{SPR}} + \mathbf{Adv}_\Sigma^{\text{EUF-CMA}}$.

Proof. An honest edge accepts a body π' for ledger entry (id, v, h, σ) only if the guard $H(\pi') = h$ evaluates true (Phase B step 7; Phase C step 12). Consider the two disjoint attack routes available to the Dolev–Yao adversary $\mathcal{A}$, who fully controls the body-fetch channel but not the endorsing organizations' signing keys.

Route 1: substitute the body, keep the ledger entry. $\mathcal{A}$ must produce $\pi' \neq \pi$ with $H(\pi') = h = H(\pi)$. Since π is fixed first (committed by the PMS), this is precisely a second preimage for H at input π; by assumption this succeeds with probability at most $\mathbf{Adv}_H^{\text{SPR}}$.

Route 2: substitute the ledger entry. $\mathcal{A}$ must cause the edge to read a tuple (id, v, h', σ') with $h' = H(\pi')$. A valid tuple requires (i) σ' to verify under the administrator public key—a forgery, probability at most $\mathbf{Adv}_\Sigma^{\text{EUF-CMA}}$—and (ii) the entry to be committed under the `AND(Org1.MSP,Org2.MSP)` endorsement, which requires signatures from both organizations' peers; a network adversary holds neither key. The edge reads the ledger through its own trusted peer connection, so $\mathcal{A}$ cannot spoof the read without breaking (i) or (ii).

A union bound over the two routes gives the claim. ■

Theorem 1 (G1, eventual consistency)

Proof. Let $D = \{id : L(id).v > w_e\}$, $N = |D|$. `getDelta`(w_e) returns every entry with version above the watermark. We show (a) completeness, (b) integrity, (c) termination. (a) *Completeness.* Suppose some $id^* \in D$ were omitted although the reply claims completeness up to the current ledger height. The reply is answered against the committed ledger state L; omission would require presenting a projection L' of L in which $L'(id^*).v \leq w_e < L(id^*).v$, i.e., presenting an earlier version as current. Because each entry's version history on PolicyChain is monotone ($\mathsf{INV}_{\text{mono}}$, enforced in chaincode at endorsement time) and the edge checks the response against ledger heights and hashes, such a rollback is detectable except with the negligible probabilities of Lemma 1. (b) *Integrity.* Each fetched body is accepted only under the hash guard; by Lemma 1 the installed body equals the PMS-signed body except with negligible probability. (c) *Termination.* The batch is applied atomically after all N verifications; by the latency decomposition of Eq. (4) of the main paper this completes within $T_{\text{rec}}(N) = a + bN$ (Corollary 1). At that point $E_e(id) = L(id)$ for all subscribed id, which is Eq. (3). ■

Corollary 1 (Reconciliation bound)

Proof. Immediate from Eq. (4) of the main paper: the constant part $a = T_{\text{order}} + T_{\text{commit}} + T_{\text{event}} + T_{\text{ack}}$ is paid once per reconnection, the linear part $b = T_{\text{fetch}} + T_{\text{verify}}$ once per missed update. E3 measures the median end-to-end latency at batch sizes 1–200; a least-squares fit of the linear regime gives $a \approx 1.0$ s and $b \approx (3558 - 1060)/(200 - 10) \approx 13$ ms/policy. ■

Theorem 2 (G2, auditability)

Proof. Existence and uniqueness. The PEP emits exactly one record ℓ per decision before returning the response (C3); ℓ enters the Offline Queue if the WAN is down. Uniqueness follows from the per-edge monotone counter seq_e incremented atomically with emission; LogChain ingestion deduplicates by (e, seq_e). *Injection.* A forged ℓ' requires a signature under the edge enrollment certificate: probability $\leq \mathbf{Adv}_{\Sigma}^{\text{EUF-CMA}}$. *Alteration.* Changing any field of a signed ℓ invalidates the signature. *Suppression.* Dropping ℓ in transit leaves a gap in seq_e, detected by the Audit & Compliance service on ingestion (the queue is drained in order); suppressing it after commit would require rewriting a committed block against the endorsement policy. ■

Theorem 3 (G3, edge-origin confidentiality)

Proof. The PEP invokes the Data Minimizer on the unique code path between the PDP's PERMIT and the network write; E6 verifies at import level that no module reachable from the response path references the raw record type. Thus

the emitted tuple is $(\psi(r.id), \gamma(r.qi), \varrho_{\text{role}}(r.sa))$. For the bound: fix the adversary's background knowledge as the target's quasi-identifier vector q^*. In a k-anonymous release, the equivalence class of q^* contains $m \geq k$ records, and the released record is exchangeable within its class from the adversary's viewpoint, so the probability of a correct link is $1/m \leq 1/k$. Over R independent releases from disjoint cohorts, failures are independent: success probability $1-(1-1/k)^R$. Pseudonyms are HMAC values under the secret study key; by the PRF property of HMAC they are computationally unlinkable across studies without the key. E9 confirms both bounds empirically within one percentage point. ■

Theorem 4 (G4, partition availability)

Proof. The edge decision path is: PEP → local PDP → Local Policy Store → Data Minimizer → response, all edge-resident; E6 includes an import-level test that the edge PDP package has no WAN-touching dependency. Log emission is absorbed by the disk-backed Offline Queue, provisioned for the maximum tolerated partition $T_{\max}$; beyond $T_{\max}$ the node enters read-only mode by design (bounded-divergence rule), which is a controlled degradation, not an availability failure within the stated bound. E5 and E12 exercise this path under forced partitions with 100% local availability. ■

Theorem 5 (G5, compliance evidence)

Proof. Given a decision d with LogChain entry $(id, s, r, a, d, v, t, \text{seq}_e)$: PolicyChain stores $(id, v) \mapsto (h, \sigma)$; the off-chain store retains the canonical body π keyed by h. The auditor recomputes $H(\pi)$ and compares with h, establishing that π is the exact text in force; the commit timestamp on PolicyChain establishes $\leq t$ ordering; σ identifies the committing administrator, supporting controller accountability (GDPR Art. 30) and the audit-trail requirements of HIPAA §164.312(b). E6's audit-replay test mechanizes this reconstruction end to end. ■

Proposition 1 (Near-optimality of the placement rule)

Proof. Let items be policies p with weight $|p|$ and value $s(p) = f(p)\Delta$, budget M. *Fractional optimality of the density order:* sort $p_1, p_2, \ldots$ by density $s(p)/|p|$ descending and fill until the budget is exhausted, splitting the boundary item. If a fractional solution x^* differed from this greedy solution x^g, there exist items i, j with density(i) >density(j), $x_i^* < x_i^g$, $x_j^* > 0$; shifting ε of weight from j to i changes the value by $\varepsilon(\text{density}(i) - \text{density}(j)) > 0$, contradicting optimality of x^*. *Integral gap:* truncating the (single) fractional boundary item loses at most $\max_p s(p)$, so integral greedy is within one policy's saving of the 0–1 optimum. *Connection to the score:* within a frequency class, Eq. (6) of the main paper ranks by $\alpha f(p) + \gamma|p|^{-1}$ with $c(p)$ as tie-breaker; with $\alpha \propto \Delta$ this ranking is monotone in $f(p)\Delta/|p|$ whenever sizes vary within a bounded factor (our compiled policies

span $< 4\times$), so the deployed order coincides with the density order on our workload. *Latency claim:* the achieved hit-rate is $\rho_{\text{edge}}(P_E) = \sum_{p \in P_E} f(p) / \sum_p f(p)$; substituting into Eq. (7) shows that maximizing $S(P_E)$ minimizes $\mathbb{E}[T_{\text{auth}}]$ over feasible P_E. E7 validates the resulting latency curve at $R^2 = 0.998$. ■

4 Policy Synchronization Diagram

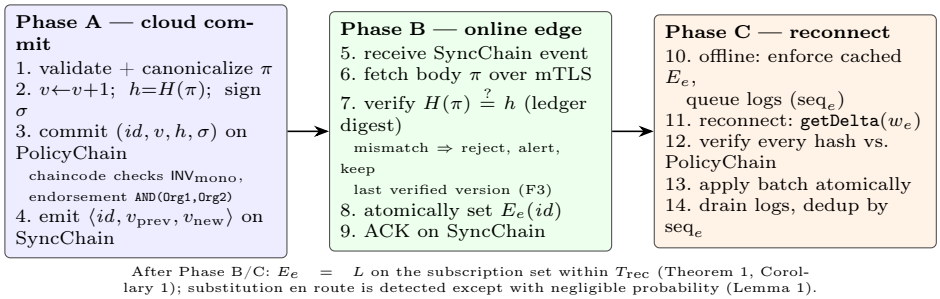


Fig. 1: Policy synchronization across PolicyChain and SyncChain: happy path (Phases A–B) and offline-to-online delta reconciliation (Phase C).

5 Data Minimizer Pipeline

6 Additional Experiment Plots

7 E12 Per-Site Table and Real-Fabric Transcript

Table 1: E12 three-site partition/soak emulation, 720 simulated minutes, 29 cloud-side policy updates (seed 20260704).

Site	Requests	Avail.	Offline dec.	Stale dec.	Partitions	Max backlog	Max missed
Urban ward gwy.	12,960	100%	270	36	4	90	1
District clinic	8,640	100%	1,740	828	12	240	2
Mobile unit	5,760	100%	1,464	400	21	128	2
Aggregate	27,360	100%	3,474	1,264	37	240	2

All 27,360 decisions were logged to LogChain with zero loss after queue drains; mean reconciliation cost was ≈ 0.01 ms per event in the in-process emulation.

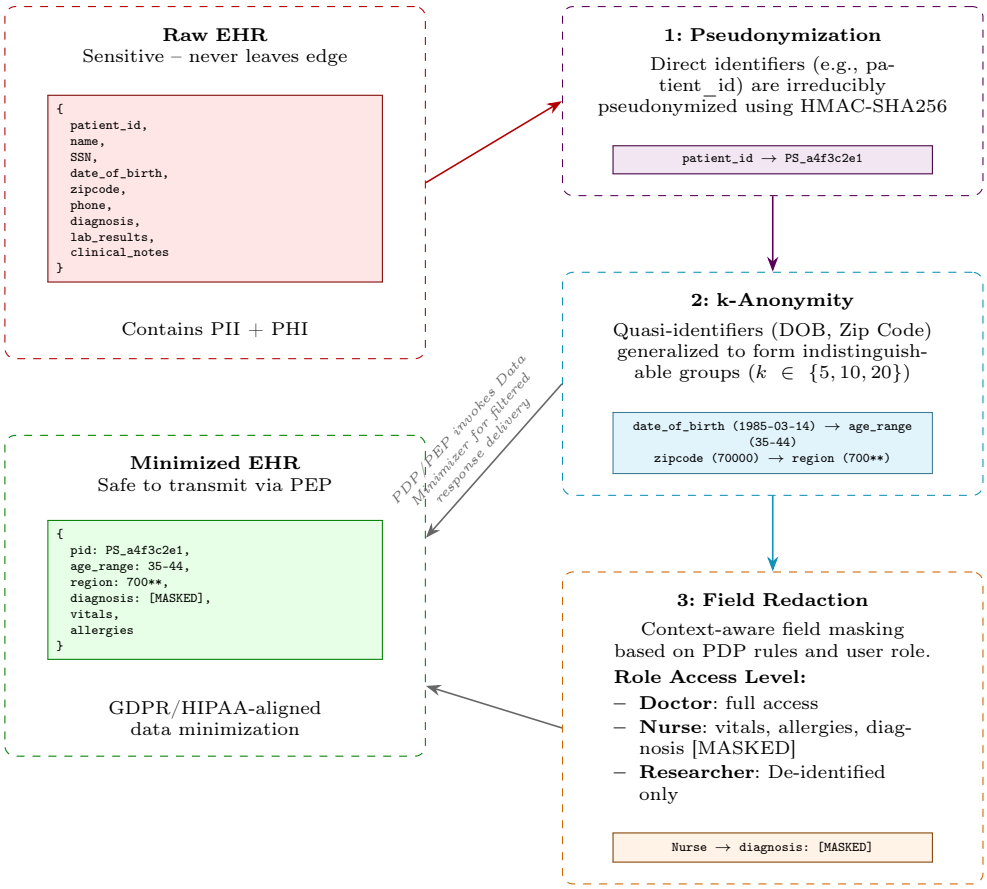
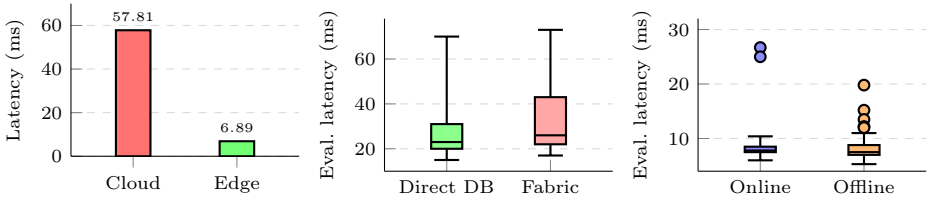
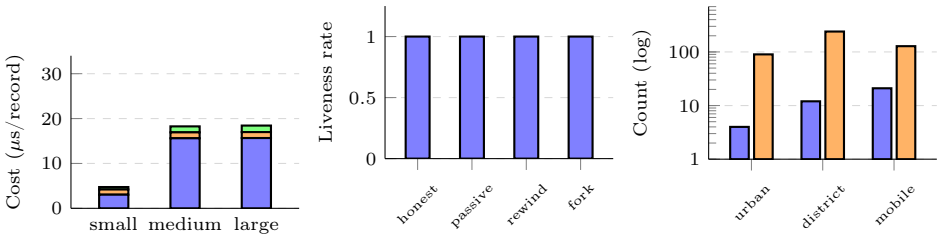


Fig. 2: Data Minimizer pipeline in the ACaaS edge layer: pseudonymization, k-anonymity, and role-based field redaction produce a minimized EHR safe for transmission via the PEP.



(a) E1: cloud vs. edge PDP (b) E4: ledger overhead (c) E5: edge autonomy

Fig. 3: Distribution plots for E1, E4, and E5. (a) E1: median authorization latency for cloud PDP (57.81 ms, 50 ms WAN emulated) vs. edge PDP (6.89 ms); (b) E4: evaluation latency boxplots for direct database write (median 23 ms) vs. Hyperledger Fabric anchoring (median 26 ms, overhead +3 ms); (c) E5: edge PDP latency distribution under online and offline (WAN-severed) conditions, showing 100% local availability with comparable latency.



(a) E8: Minimizer per-stage cost (b) E11: liveness (0 $\mathsf{INV}_{\text{mono}}$ violations in 200 commits) (c) E12: per-site partitions and backlog

Fig. 4: Companion plots: (a) E8 Data Minimizer per-stage median cost (pseudonymize, generalize, redact); (b) E11 liveness under Byzantine behaviours; (c) E12 per-site partition count and peak queue backlog (100% availability at all sites).

Stale decisions (4.6% of total) are decisions taken against a superseded but hash-verified policy version during a partition; all were reconciled by Phase C.

Real-Fabric transcript summary. The official Hyperledger Fabric test network (two organizations, Raft ordering) was deployed with a JavaScript policy-hash chaincode on channel `acaaschan`: 8/8 `invoke_create` transactions succeeded with median 164 ms and P95 191 ms; 8/8 ledger reads returned within the retry window with median 137 ms; final channel height was 16. This run is a feasibility check for the latency constants assumed in Eq. (4) of the main paper, not a re-run of the mock-based E10/E11 sweeps.

8 Reproduction Instructions

The artefact is available at https://github.com/SonHaXuan/ACaaS and was last executed with Python 3.12.3.

- `pytest -q` — runs the full test suite (54 tests, including the 21-assertion G1–G5 conformance suite under `tests/security/`); completes in seconds.
- `python -m benchmarks.run_all` — regenerates every E6–E12 figure (PDF) and the underlying JSON result files under `results/`.
- `docker-compose up` — brings up the full cloud–fog–edge stack on a single host for interactive exploration.
- `results/manifest.sha256` — checksums of the result bundle used in the paper.